



XenSense-V1

Real-time video perception for autonomous driving

Part 2 of 2 — **Codebase & File Guide**

What every module does, and how a frame flows through the pipeline.

Prepared 14 July 2026 · Author: Bhanu Prakash

o • The folder map

A single Python package (`xensense/`) orchestrated by `main.py`. Each capability is its own module, so any of them can be toggled independently at runtime.

```
Research/ (XenSense-V1)
├─ main.py                # CLI entry: argparse → Config → run pipeline
├─ requirements.txt       # ultralytics, opencv, torch, deep-sort-realtime...
├─ REPORT.pdf             # the underlying research report
├─ yolov8n-seg.pt         # default YOLOv8 nano segmentation weights
├─ xensense/
│   ├─ config.py          # all thresholds, class filter, size priors
│   ├─ pipeline.py        # orchestrator: per-frame processing loop
│   └─ modules/
│       ├─ preprocessing.py # VideoReader + Frame (resize, denoise)
│       ├─ segmentation.py  # YOLOv8-seg wrapper → Detection
│       ├─ tracking.py      # DeepSORT + IoU fallback
│       ├─ memory.py        # occlusion ghosts + histogram re-ID
│       ├─ analytics.py     # speed / direction / distance + optical flow
│       ├─ smoke_detection.py # CNN or classical smoke/fog
│       ├─ pothole_detection.py # YOLO or classical dark-blob
│       ├─ renderer.py      # draws boxes, masks, trails, HUD
│       └─ exporter.py      # CSV rows + JSON summary
│   └─ models/smoke_cnn.py  # ~120k-param SmokeCNN
├─ scripts/ train_smoke.py, make_test_video.py
└─ weights/ data/ logs/ out/ # weights (optional), inputs, logs, outputs
```

Per-frame data flow: `VideoReader` → `Segmenter` (YOLOv8-seg) → `Tracker` (IDs) → `MemoryBank` (ghosts + re-ID) → `ObjectAnalytics` (speed/dir/dist) — in parallel with `SmokeFogDetector`, `PotholeDetector` and optional `OpticalFlowEstimator` — → `Renderer` (composite + HUD) → `ResultExporter` (CSV/JSON).
Nothing runs a module the toggles have turned off.

1 • The pipeline core

ENTRY POINT

`main.py`

Parses CLI args (`--source` , `--output` , `--model` , `--conf` , `--width` , `--device` , `--max-frames` , module toggles), builds a `Config` , and runs `XenSensePipeline` .

CONFIGURATION

`xensense/config.py`

`Config` + `ModuleToggles` dataclasses: all thresholds, the driving-relevant class filter, device selection, and the real-world size priors (`CLASS_WIDTHS_M` , `CLASS_HEIGHTS_M`) used for monocular distance.

ORCHESTRATOR

`xensense/pipeline.py`

Builds every module, runs the main loop (`run()` → `process_frame()`), handles keyboard toggles, video writing and the final JSON summary.

2 • The modules (`xensense/modules/`)

FILE	PURPOSE
<code>preprocessing.py</code>	<code>VideoReader</code> + <code>Frame</code> dataclass — capture, resize, light denoise, grayscale copy, timestamp/index.
<code>segmentation.py</code>	<code>Segmenter</code> (YOLOv8-seg wrapper) + <code>Detection</code> dataclass (bbox, conf, class, mask, track_id, ghost, analytics).
<code>tracking.py</code>	<code>DeepSortTracker</code> , <code>IoUTracker</code> fallback, <code>build_tracker()</code> — persistent IDs.
<code>memory.py</code>	<code>MemoryBank</code> + <code>MemoryEntry</code> — occlusion prediction (ghost boxes) and HSV-histogram re-identification.
<code>analytics.py</code>	<code>ObjectAnalytics</code> (speed/direction/distance/trail) and <code>OpticalFlowEstimator</code> (Farnebäck flow).
<code>smoke_detection.py</code>	<code>SmokeFogDetector</code> — CNN or classical colour/drift/texture backend + fog cue + temporal voting.
<code>pothole_detection.py</code>	<code>PotholeDetector</code> — YOLO or classical dark-blob backend + shadow suppression + temporal persistence.
<code>renderer.py</code>	<code>Renderer</code> — draws masks, boxes, labels, trails, potholes, hazard banners, ghost boxes, and the HUD.
<code>exporter.py</code>	<code>ResultExporter</code> — CSV writer + JSON summary (class/unique counts, hazard ranges, FPS).

3 · Models, scripts & data

FILE / FOLDER	PURPOSE
<code>xensense/models/smoke_cnn.py</code>	The ~120k-parameter <code>SmokeCNN</code> ($3 \times 128 \times 128 \rightarrow$ single logit) used when trained weights are present.
<code>scripts/train_smoke.py</code>	Trains the SmokeCNN on a <code>smoke/</code> + <code>clear/</code> dataset $\rightarrow$ <code>weights/smoke_cnn.pt</code> .
<code>scripts/make_test_video.py</code>	Generates the synthetic test clip (moving shapes, drifting smoke plume, dark potholes) to exercise the classical modules.
<code>weights/</code>	Target dir for optional trained weights — currently empty (classical fallbacks active).
<code>data/</code> · <code>logs/</code> · <code>out/</code>	Input videos · per-run CSV+JSON logs · annotated output videos and result frames.

4 · Commands

COMMAND	WHAT IT DOES
<code>pip install -r requirements.txt</code>	Install deps (YOLO weights auto-download on first run).
<code>python main.py --source data/test.mp4 --output out/annotated.mp4</code>	Run on a video, save the annotated result.
<code>python main.py --source 0</code>	Run on the webcam.
<code>python main.py --source v.mp4 --no-display --all-classes</code>	Headless run over all 80 COCO classes.

Live keyboard toggles (preview focused): `d` detection · `o` masks · `t` tracking · `m` memory · `s` smoke · `p` pothole · `a` analytics · `f` optical flow · `r` trails · `h` HUD · `q` quit. CLI start-state flags: `--no-smoke` , `-no-pothole` , `--no-tracking` , `--all-classes` , `--flow` , `--no-display` .